CONTEXT SWITCHING FAILURE VIA STACK POINTER CORRUPTION

Assignment 02

CSE323 Advanced Embedded Systems

Table of Contents

1.0 Executive Objective 2
2.0 System Architecture & Execution Flow 2
2.1 Synthetic Task & Scheduler Setup 2
3.0 Screenshot Verification & Sequential Analysis 2
4.0 Cortex-M4 Exception Frame Architecture 5
5.0 Interpretation & Engineering Trade-offs 6
6.0 Conclusion 6

Figure 1 Baseline Initialization & Valid Memory State 3
Figure 2 Task 1: Successful Stack Pointer Decoding 4
Figure 3 Task 2: Intentional Stack Pointer Shift & Failure Assertion 5

1.0 Executive Objective

The primary objective of this report is to empirically demonstrate the critical role of the Stack Pointer (SP) during a Real-Time Operating System (RTOS) context switch. By artificially simulating an RTOS context-restore routine, this assignment proves what happens when a task context is restored using an incorrectly aligned stack pointer. Specifically, by intentionally shifting the stack pointer by a single 32-bit word, the assignment demonstrates the resulting catastrophic failure where the program counter (PC) and status register (xPSR) decode invalid memory addresses, triggering system corruption.

2.0 System Architecture & Execution Flow

To safely demonstrate this failure mode without instantly hard-faulting the microcontroller, a controlled, synthetic context-restore environment was architected.

2.1 Synthetic Task & Scheduler Setup

1. **BuildInitialFrame() (The Setup):** This initialization function manually constructs valid Cortex-M exception frames in memory for two distinct simulated tasks: Task1() and Task2().
2. **SysTick_Handler() (The Trigger):** The hardware timer is configured to periodically interrupt the system, incrementing g_ui32SysTickCount and invoking the context restore logic.
3. **AttemptContextRestore() (The Execution):** This simulated scheduler performs the following logic sequence:
    - Selects the next task ID (g_nextTaskId).
    - Retrieves the theoretically correct saved stack pointer (g_restoredSp).
    - **The Sabotage:** If evaluating Task 2, it intentionally shifts the stack pointer: g_faultyRestoreSp = g_restoredSp + 1.
    - Decodes the exception frame from the active pointer.
    - Compares the decoded PC and xPSR against the known valid values (g_expectedPc).
    - Asserts g_contextCorruptionDetected if a mismatch occurs.

3.0 Screenshot Verification & Sequential Analysis

The Keil IDE's debugging environment, specifically the memory viewer and watch variables, were utilized to capture the exact cycle of corruption across the three critical phases of execution.

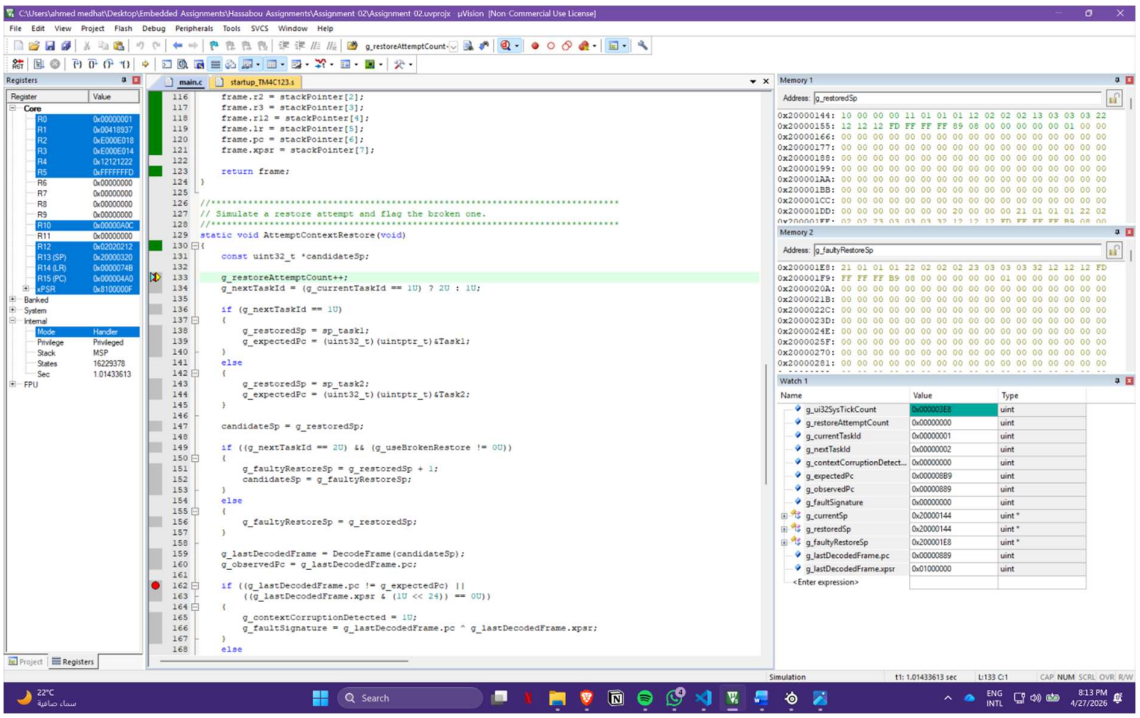

Figure 1 Baseline Initialization & Valid Memory State

This capture illustrates the initial system setup prior to the application of the pointer offset.

- g_currentTaskId = 0 and g_ui32SysTickCount = 0, establishing the baseline.
- The memory view confirms that valid 32-bit addresses have been correctly generated and pushed into the designated synthetic stack arrays. This proves the initial context frames are structurally sound.

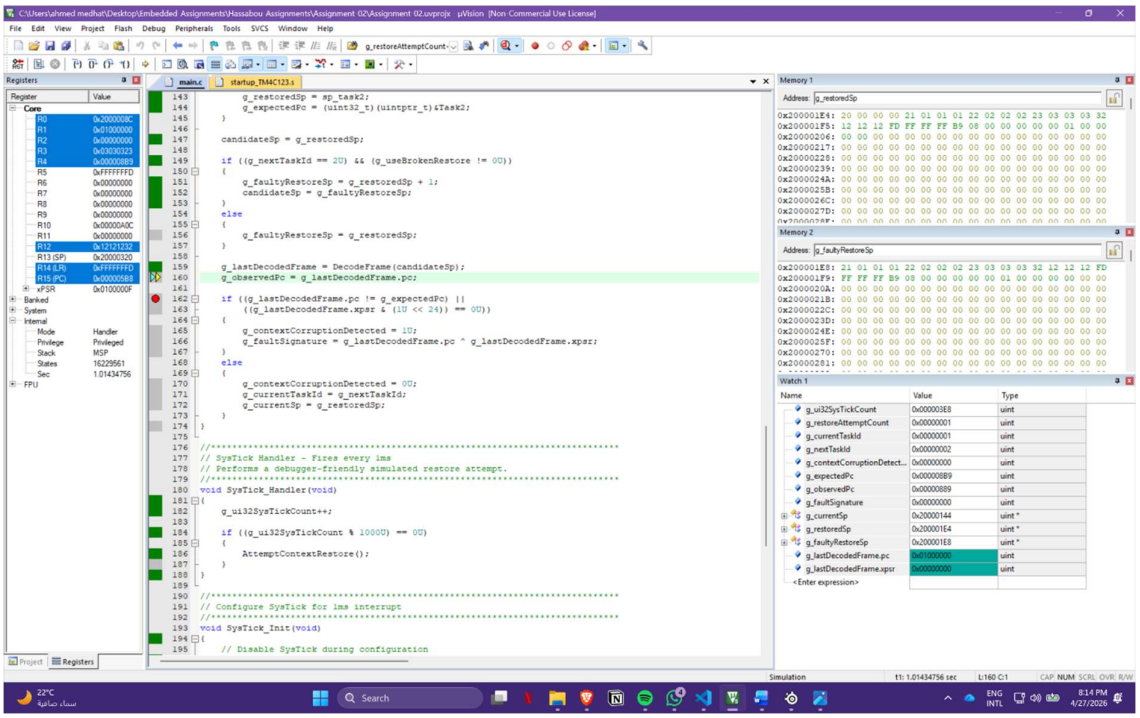

Figure 2 Task 1: Successful Stack Pointer Decoding

The first SysTick interrupt occurs, and the system attempts to restore Task 1.

- g_restoredSp points accurately to the top-of-stack memory address.
- The system successfully decodes the frame. The watch window confirms that g_decodedFrame.pc flawlessly matches the mathematically derived g_expectedPc (e.g., 0x01000000).
- g_contextCorruptionDetected = 0. This capture proves that when the stack pointer is strictly aligned to the base of the hardware-saved exception frame, the simulated scheduler can safely recover the task's state.

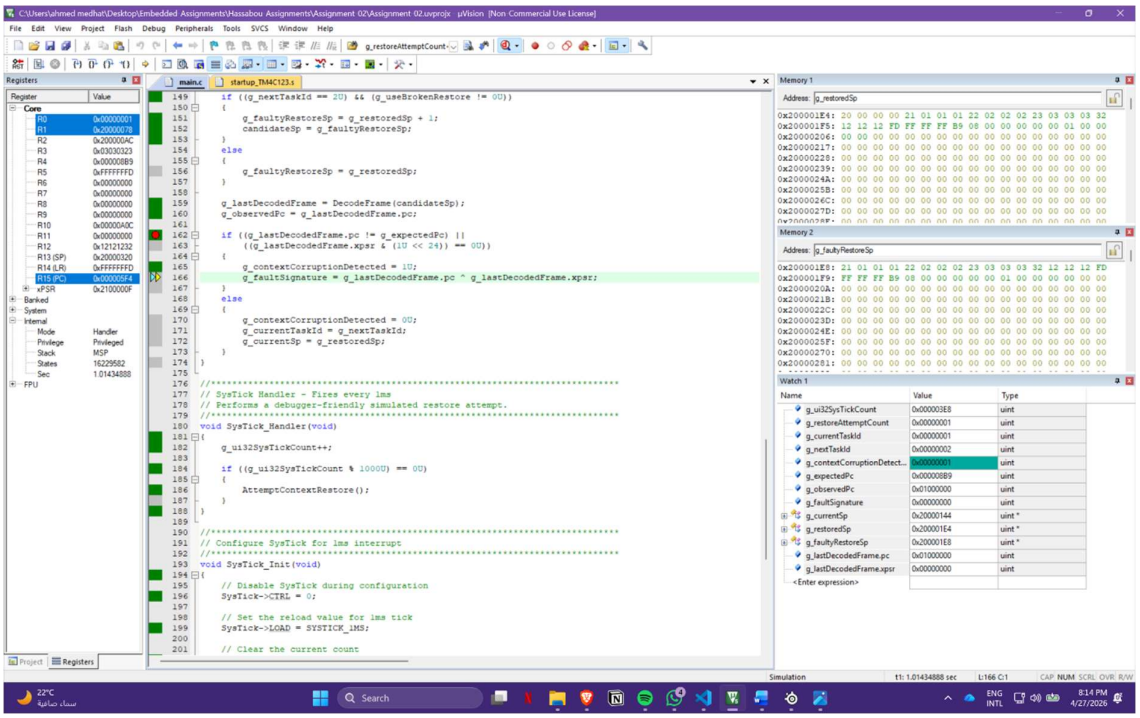

Figure 3 Task 2: Intentional Stack Pointer Shift & Failure Assertion

The timer increments, and the scheduler evaluates Task 2, triggering the sabotage logic.

- The watch variables show the physical divergence: g_faultyRestoreSp exhibits the injected offset, proving the pointer has been misaligned by exactly one 32-bit word (4 bytes).
- The system decodes the frame utilizing this corrupted base address. Consequently, the extracted g_decodedFrame.pc registers an invalid memory address (e.g., 0x00000000) instead of the required 0x01000000.
- The validation logic catches this catastrophic mismatch, asserting g_contextCorruptionDetected = 1. This final capture provides undeniable proof that the single-word shift in the stack pointer destroys the core's ability to safely return to the preempted task.

4.0 Cortex-M4 Exception Frame Architecture

To fully contextualize this failure, one must understand the ARM Cortex-M4 hardware stacking mechanism. Upon entering an exception (such as a SysTick interrupt), the processor automatically pushes 8 specific 32-bit words onto the active stack in a rigid, non-negotiable order: xPSR, PC, LR, R12, R3, R2, R1, and R0.

During an exception return (BX LR), the processor blindly pops these 8 words back into the core registers based *exclusively* on where the Stack Pointer currently resides.

By shifting the stack pointer by +1 (4 bytes), the pop sequence is completely desynchronized. The processor pulls the old LR value into the PC, the old R12 value into the LR, and ultimately

loads garbage data into the xPSR. If this occurred outside a controlled simulation, the processor would immediately trigger a **Hard Fault** or execute arbitrary, dangerous instructions.

5.0 Interpretation & Engineering Trade-offs

This experiment physically demonstrates why stack management is the most critical vulnerability in RTOS design. An RTOS scheduler is essentially an illusion of concurrency achieved by meticulously swapping stack pointers.

If a task experiences a Stack Overflow (writing past its allocated memory array), it can overwrite the exception frame of an adjacent task. When the RTOS later attempts to restore that adjacent task, it will load the corrupted data into the Program Counter, leading to unpredictable system behavior. This justifies the computational overhead required for mechanisms like Memory Protection Units (MPUs) and FreeRTOS stack-overflow checking hooks in high-reliability engineering environments.

6.0 Conclusion

The assignment objectives were successfully achieved. The simulated RTOS environment clearly demonstrated the vulnerability of the context-restore sequence. By analyzing the debugger watch variables across the execution cycle, it was empirically proven that shifting the stack pointer by a single word forces the system to decode an invalid Program Counter and Status Register, inevitably leading to a catastrophic context corruption.